

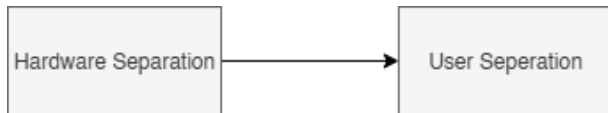
A Journey to Kubernetes

Yannik Schmidt

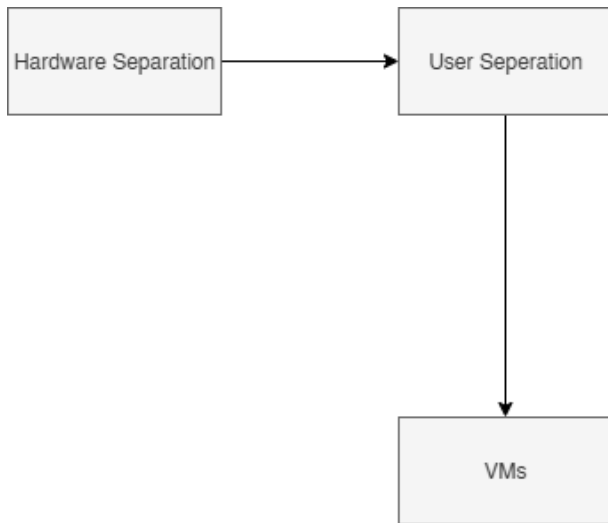
Lightning Talks @FAU

Docker

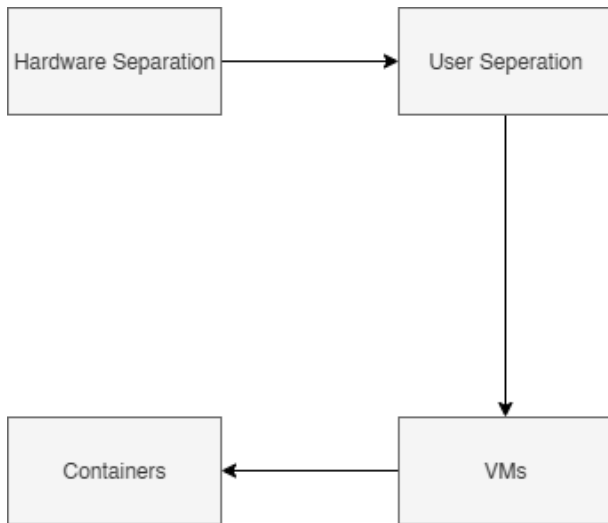
VMs & Docker



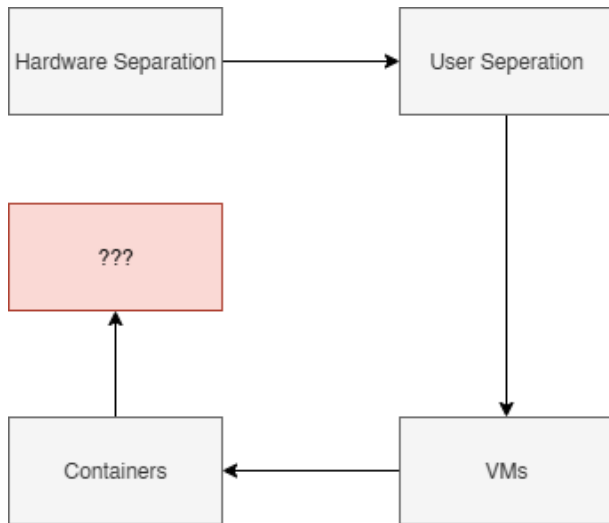
VMs & Docker



VMs & Docker



VMs & Docker



Why not VMs?

- ▶ VM images might be big

Why not VMs?

- ▶ VM images might be big
- ▶ VM builds might be complicated

Why not VMs?

- ▶ VM images might be big
- ▶ VM builds might be complicated
- ▶ significant overhead

Why not VMs?

- ▶ VM images might be big
- ▶ VM builds might be complicated
- ▶ significant overhead
- ▶ getting & running VM images might be annoying

Solving these Problems with Docker

- ▶ Docker Engine runs containerized, isolated applications

Solving these Problems with Docker

- ▶ Docker Engine runs containerized, isolated applications
- ▶ standardized **Dockerfile** syntax defines "VM" Images

Solving these Problems with Docker

- ▶ Docker Engine runs containerized, isolated applications
- ▶ standardized **Dockerfile** syntax defines "VM" Images
- ▶ a Docker client pulls, configures & runs the images

Dockerfile

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

requirements.txt

```
flask >= 3.12
```

Dockerfile

Dockerfile

```
FROM python:3.10-alpine
WORKDIR /app
COPY requirements.txt requirements.txt
COPY app.py app.py
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["flask", "run"]
```

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

requirements.txt

```
flask >= 3.12
```

Dockerfile

Dockerfile

```
FROM python:3.10-alpine
```

```
WORKDIR /app
```

```
COPY requirements.txt requirements.txt
```

```
COPY app.py app.py
```

```
RUN pip install -r requirements.txt
```

```
EXPOSE 5000
```

```
CMD ["flask", "run"]
```

Application Image

A horizontal arrow points from the Dockerfile code block to a rectangular box labeled "Application Image".

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

requirements.txt

```
flask >= 3.12
```


From Docker to Kubernetes

From Docker to Kubernetes

```
docker run --name app-container
  -p 5000:5000
  -v ./data/app/data
  -e "LOG_LEVEL: debug"
  -e "DB_USER: ..."
  -e "DB_PASS: ..."
  app-image
```

From Docker to Docker Compose to Kubernetes

docker-command

docker run

--name app-container

-p 5000:5000

-v ./data:/app/data/

-e LOG_LEVEL: debug

-e DB_USER: ...

-e DB_PASS: ...

app-image

docker-compose.yml

service:

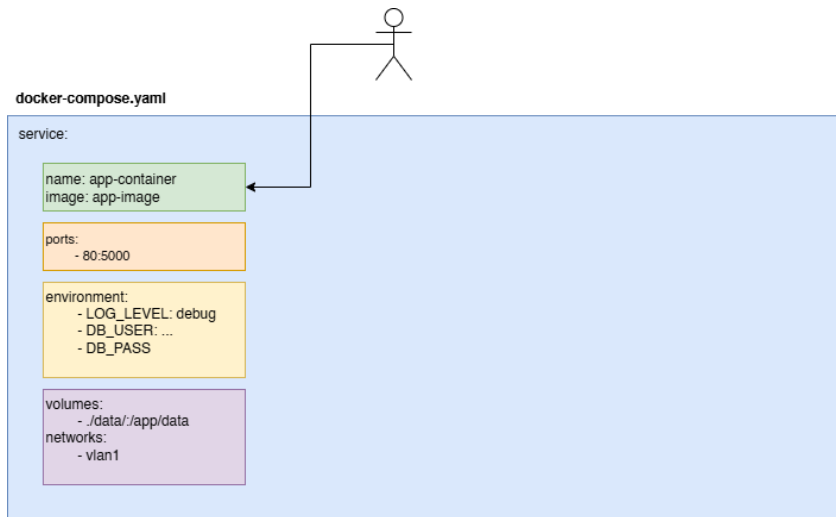
name: app-container
image: app-image

ports:
- 5000:5000

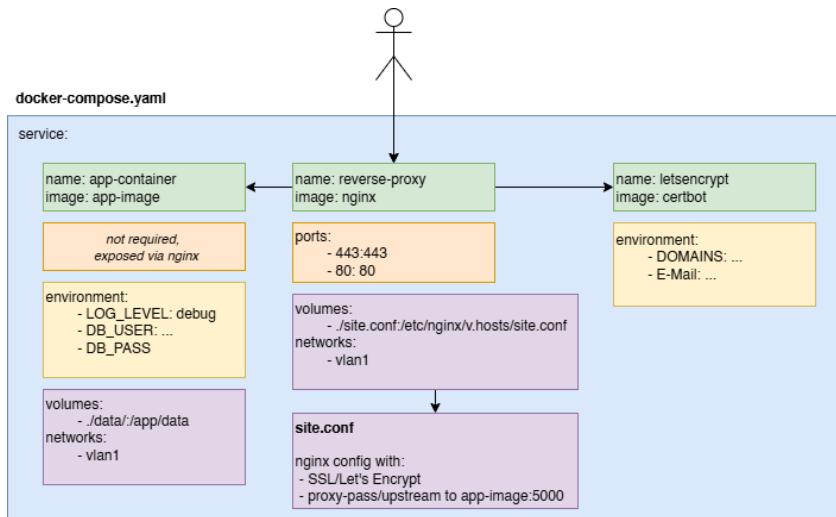
environment:
- LOG_LEVEL: debug
- DB_USER: ...
- DB_PASS

volumes:
- ./data:/app/data
networks:
- vlan1

From Docker to Docker Compose to Kubernetes



From Docker to Docker Compose to Kubernetes



Management has some requirements

- ▶ ..is something nobody ever wants to hear

Management has some requirements

- ▶ load distribution over multiple machines/nodes

Management has some requirements

- ▶ load distribution over multiple machines/nodes
- ▶ secure communication between nodes

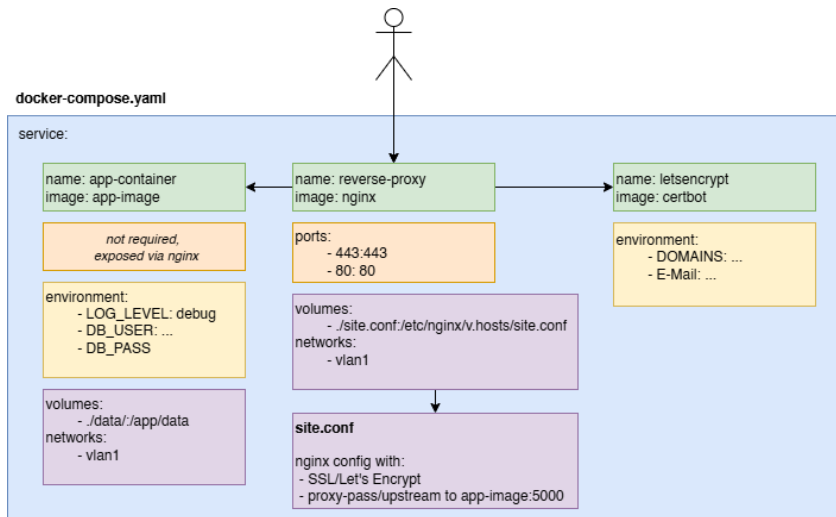
Management has some requirements

- ▶ load distribution over multiple machines/nodes
- ▶ secure communication between nodes
- ▶ an HTTP API to manage the cluster?

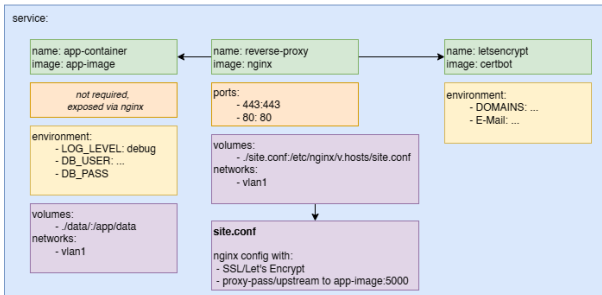
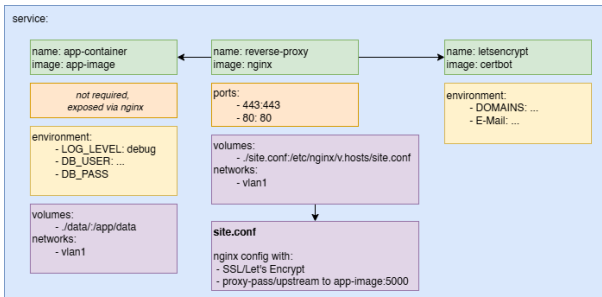
Management has some requirements

- ▶ load distribution over multiple machines/nodes
- ▶ secure communication between nodes
- ▶ an HTTP API to manage the cluster?
- ▶ easier TLS management

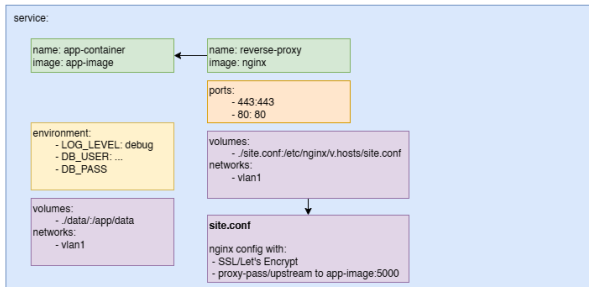
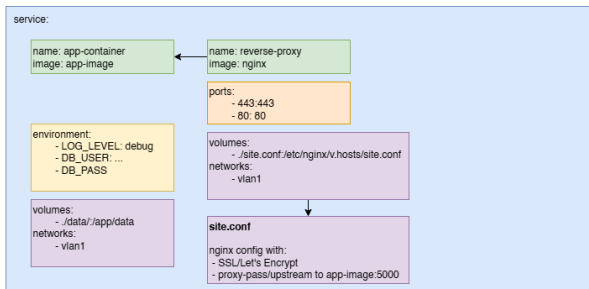
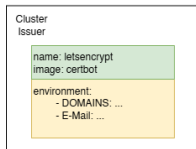
From Docker to Docker Compose to Kubernetes



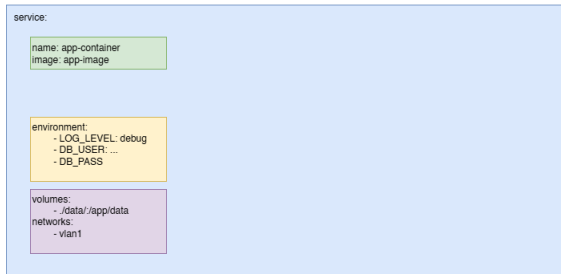
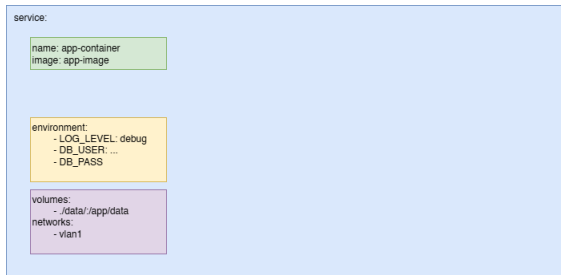
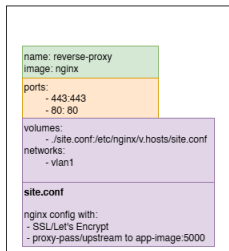
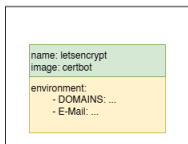
From Docker to Docker Compose to Kubernetes



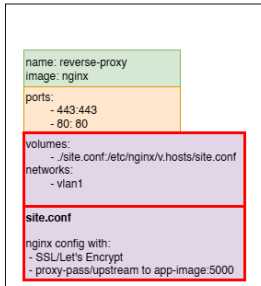
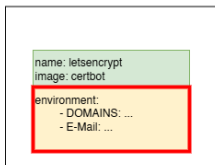
From Docker to Docker Compose to Kubernetes



From Docker to Docker Compose to Kubernetes



From Docker to Docker Compose to Kubernetes



service:

```
name: app-container
image: app-image
```

environment:

- LOG_LEVEL: debug
- DB_USER: ...
- DB_PASS

volumes:

- /data:/app/data

networks:

- vlan1

service:

```
name: app-container
image: app-image
```

environment:

- LOG_LEVEL: debug
- DB_USER: ...
- DB_PASS

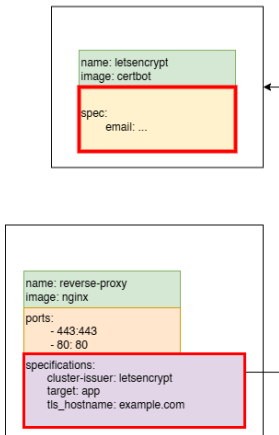
volumes:

- /data:/app/data

networks:

- vlan1

From Docker to Docker Compose to Kubernetes



service:

```
name: app-container
image: app-image
```

environment:

- LOG_LEVEL: debug
- DB_USER: ...
- DB_PASS

volumes:

- /data:/app/data

networks:

- vlan1

service:

```
name: app-container
image: app-image
```

environment:

- LOG_LEVEL: debug
- DB_USER: ...
- DB_PASS

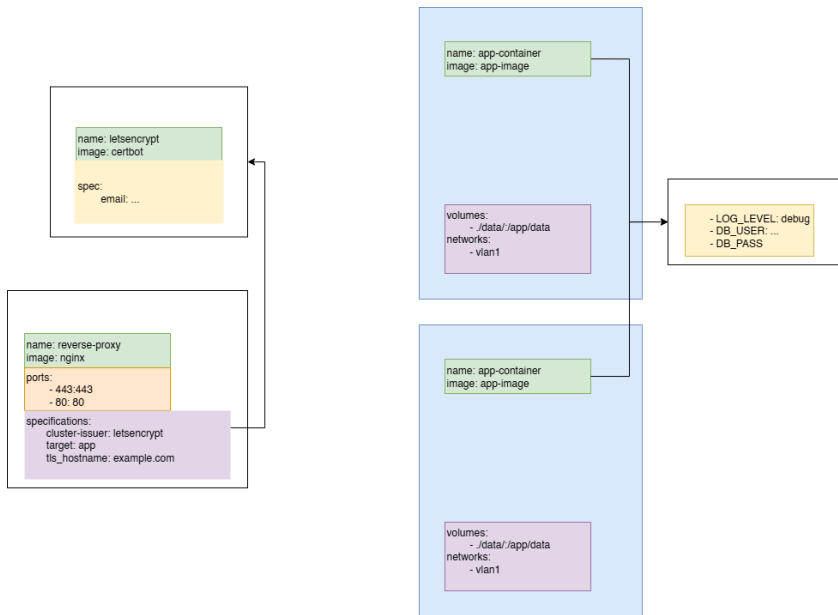
volumes:

- /data:/app/data

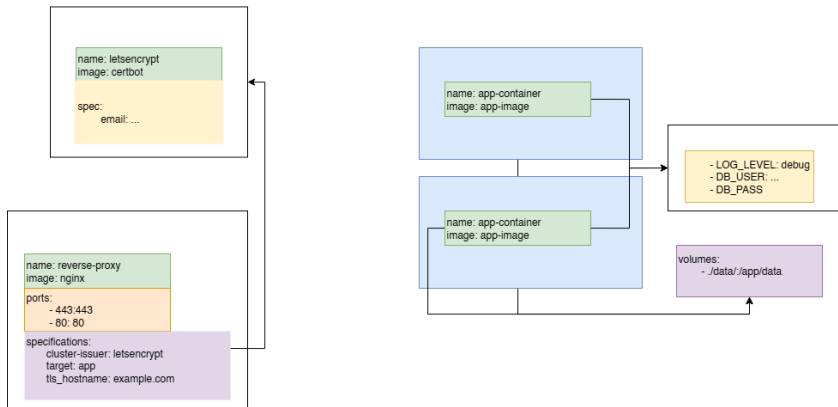
networks:

- vlan1

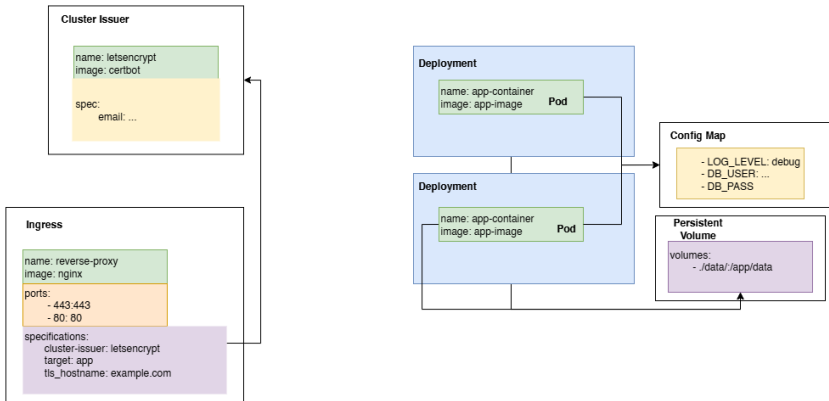
From Docker to Docker Compose to Kubernetes



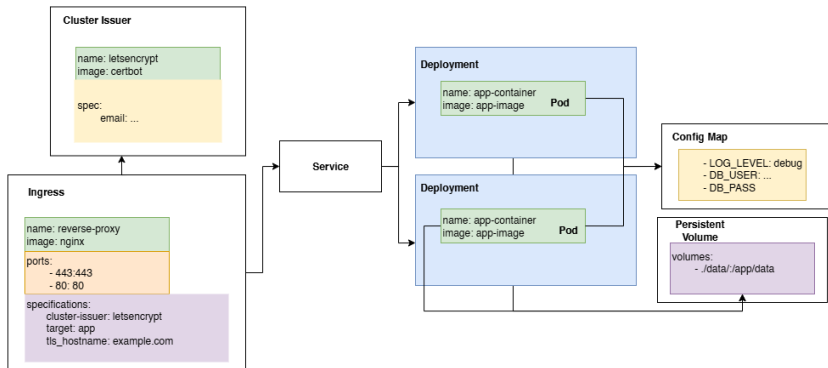
From Docker to Docker Compose to Kubernetes



From Docker to Docker Compose to Kubernetes



From Docker to Docker Compose to Kubernetes



Kubernetes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
  labels:
    app: app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: app
          imagePullPolicy: Always
          image: app-image
          ports:
            - containerPort: 5000
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  labels:
    app: app
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    kubernetes.io/ingress.class: "nginx"
spec:
  tls:
    - hosts:
        - app.example.com
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app
                port:
                  number: 80
```

```
apiVersion: v1
kind: Service
metadata:
  name: app
  labels:
    app: app
spec:
  selector:
    app: app
  ports:
    - name: http
      port: 80
      targetPort: 5000
  type: ClusterIP
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_USER: ...
  OTHER: ...
```

Kubernetes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
  labels:
    app: app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: app
          imagePullPolicy: Always
          image: app-image
          ports:
            - containerPort: 5000
```

```
apiVersion: v1
kind: Service
metadata:
  name: app
  labels:
    app: app
spec:
  selector:
    app: app
  ports:
    - name: http
      port: 80
      targetPort: 5000
  type: ClusterIP
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  labels:
    app: app
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    kubernetes.io/ingress.class: "nginx"
spec:
  tls:
    - hosts:
        - app.example.com
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app
                port:
                  number: 80
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_USER: ...
  OTHER: ...
```

Why do Kubernetes Projects Fail

Kubernetes

- ▶ ..automatically scales stateless container loads

Kubernetes

- ▶ ..automatically scales stateless container loads
- ▶ ..automatically handles certificates

Kubernetes

- ▶ ..automatically scales stateless container loads
- ▶ ..automatically handles certificates
- ▶ ..has great UIs for semi-technical operators

Kubernetes

- ▶ ..automatically scales stateless container loads
- ▶ ..automatically handles certificates
- ▶ ..has great UIs for semi-technical operators
- ▶ ..automatically encrypts & handles internal traffic

Kubernetes

- ▶ ..automatically scales stateless container loads
- ▶ ..automatically handles certificates
- ▶ ..has great UIs for semi-technical operators
- ▶ ..automatically encrypts & handles internal traffic
- ▶ ..has great automatic deployment options (GitOps)

Kubernetes also

- ▶ ..hard to maintain by a single person

Kubernetes also

- ▶ ..hard to maintain by a single person
- ▶ ..very expensive if you don't self-host it

Kubernetes also

- ▶ ..hard to maintain by a single person
- ▶ ..very expensive if you don't self-host it
- ▶ ..does not make your application magically HA (Highly Available)

Kubernetes also

- ▶ ..hard to maintain by a single person
- ▶ ..very expensive if you don't self-host it
- ▶ ..does not make your application magically HA (Highly Available)
- ▶ ..does not allow easy manual intervention

Kubernetes also

- ▶ ..hard to maintain by a single person
- ▶ ..very expensive if you don't self-host it
- ▶ ..does not make your application magically HA (Highly Available)
- ▶ ..does not allow easy manual intervention
- ▶ ..requires you to think in certain frameworks

Kubernetes also

- ▶ ..hard to maintain by a single person
- ▶ ..very expensive if you don't self-host it
- ▶ ..does not make your application magically HA (Highly Available)
- ▶ ..does not allow easy manual intervention
- ▶ ..requires you to think in certain frameworks
- ▶ ..is worse the less stateless your Pod is

Avoid All-in-One Obsession

- ▶ Database in Kubernetes → very hard to get right, don't do it

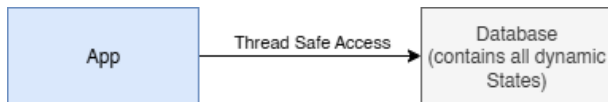
Avoid All-in-One Obsession

- ▶ Database in Kubernetes → very hard to get right, don't do it
- ▶ Container Registry in Kubernetes → please don't

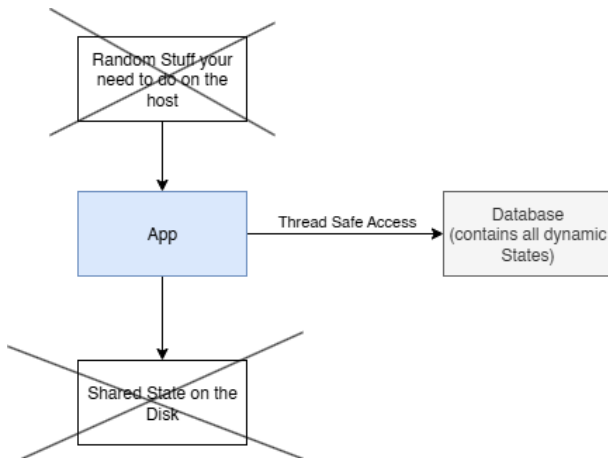
Avoid All-in-One Obsession

- ▶ Database in Kubernetes → very hard to get right, don't do it
- ▶ Container Registry in Kubernetes → please don't
- ▶ Git in Kubernetes → just why

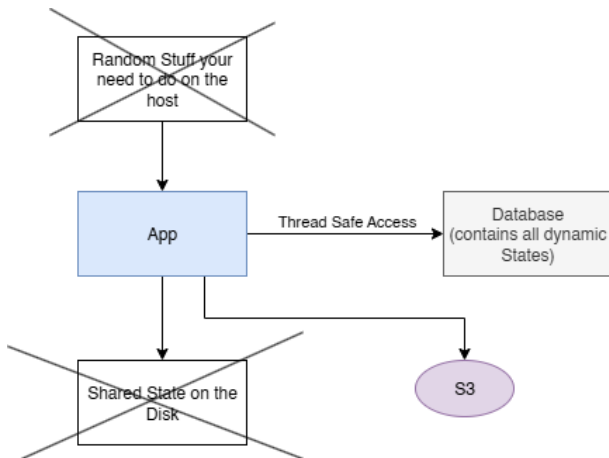
What should your app look like to work well in Kubernetes



What should your app look like to work well in Kubernetes



What should your app look like to work well in Kubernetes



Fragen? Nächster Vortrag

- ▶ **Nächster Vortrag:** *TODO*
- ▶ **Fragen**